

Banking Web Application Teaching Guide

Part 2: Authentication, JWT, Spring Security, and Core Backend Annotations

This part focuses on the security heart of the project. If you can explain this clearly, you will sound like someone who understands not just coding, but safe system design.

1. Authentication Module Overview

Purpose: The authentication module proves who the user is. In this project, it handles registration, login, password encryption, token generation, and identity retrieval for future requests.

Main classes involved: `AuthController`, `AuthService`, `CustomUserDetailsService`, `JwtService`, `JwtAuthenticationFilter`, and `SecurityConfig`.

Module interaction diagram:

```
Login Request
-> AuthController
-> AuthService
-> AuthenticationManager
-> CustomUserDetailsService
-> UserRepository / MySQL
-> JwtService generates token
-> JSON response back to React
```

Why this code is written this way: login should not be handled by one giant class. We separate HTTP entry, business logic, token generation, and user lookup so each piece has one clear responsibility.

Viva line: "Authentication is split into controller, service, and security components so that user verification, password checking, and token generation stay modular and maintainable."

Mentor note: in interviews, always separate authentication from authorization. Authentication answers 'who are you?' Authorization answers 'what are you allowed to do?'

2. JWT Flow Visually

Purpose: JWT allows stateless authentication. After login, the server does not need to store a session in memory. Instead, the client sends the token with each request.

Simple JWT lifecycle:

1. User logs in with email + password
2. Backend verifies credentials
3. Backend generates JWT token
4. Frontend stores token
5. Frontend sends token in Authorization header
6. Backend validates token on each protected request
7. Backend allows or rejects access

Visual flow diagram:

```
React Login Page
|
| POST /api/auth/login
v
Spring Boot
|
| validate username/password
| create JWT
v
React stores JWT
|
| GET /api/customer/account
| Authorization: Bearer <token>
v
JwtAuthenticationFilter
|
| token valid? -> yes
v
Controller / Service / DB
|
v
JSON Response
```

Why JWT is useful here: It works well for REST APIs, React frontends, and role-based access. It also scales better than tightly coupled server sessions.

Security flow: the JWT contains the username and role claims. The backend verifies the signature before trusting any token data. If the signature is invalid or the token is expired, access is denied.

Frontend-backend communication: Axios automatically attaches the JWT from local storage in the Authorization header. This keeps protected API calls consistent across pages.

Database interaction: the token itself is not stored in MySQL in this version. Instead, once a token is validated, the user is loaded again from the database using email. That ensures current user status and roles remain trusted from server-side data.

3. Spring Security Configuration, Explained Simply

Purpose: Spring Security protects the application before requests reach business logic. It defines which endpoints are public, which require login, and which roles can access them.

Mental model:

Public endpoints:

/api/auth/**

Admin only endpoints:

/api/admin/**

Customer or Admin endpoints:

/api/customer/**

In **SecurityConfig**, we disable CSRF for this stateless REST API, enable CORS for the React app, set the session policy to **STATELESS**, define authorization rules, and insert the JWT filter before the normal username-password authentication filter.

Simple filter chain idea:

HTTP Request

-> CORS check

-> JWT filter

-> role check

-> controller method

-> response

Why written this way: placing JWT validation in a filter means every protected request is checked centrally instead of repeating token logic in every controller.

Viva line: "Spring Security is configured as a stateless API security layer. Public authentication routes are open, while customer and admin APIs are protected by role-based authorization rules."

Mentor note: if someone asks why sessions are disabled, say because JWT-based APIs are stateless and do not rely on server-side session tracking.

4. Important Spring Annotations You Should Explain in Viva

Purpose: Annotations tell Spring how to wire the application. A junior developer should not just memorize them, but understand what each one does in the architecture.

Annotation	Meaning	Where used in this project
@SpringBootApplication	Main boot entry point	BankingApplication.java
@RestController	Class returns JSON responses	AuthController, CustomerController, AdminController
@RequestMapping	Base URL path mapping	API route grouping
@GetMapping / @PostMapping / @PutMapping / @DeleteMapping	HTTP method mapping	REST endpoints
@Service	Business logic bean	Transfer, auth, fraud, admin services
@Repository	Persistence component	JPA repositories
@Entity	Maps class to database table	User, Account, Transaction, FraudLog
@Table	Customizes table name	All entity classes
@Id / @GeneratedValue	Primary key mapping	Entity identifiers
@ManyToOne / @OneToMany / @ManyToMany	Relationship mapping	User-account-role-transaction relations
@Valid	Triggers validation on request DTOs	Register, login, transfer, profile update
@Transactional	Run logic safely as one DB transaction	Register and transfer logic
@EnableMethodSecurity	Enables method-level security support	SecurityConfig
@RequiredArgsConstructor	Generates constructor injection	Services and controllers

Why these matter: Annotations reduce boilerplate, make the code declarative, and clearly communicate developer intent. In enterprise Java, being able to explain annotations is a big signal of competence.

5. Security Flow Module by Module

AuthController

Purpose: receives login and registration HTTP requests.

Request flow: validates JSON input, forwards it to AuthService, returns token response.

Why written this way: controllers should stay thin and not contain password or token logic.

AuthService

Purpose: handles registration and login business logic.

Security flow: encrypts passwords with BCrypt, authenticates credentials through Spring Security, and creates JWT tokens.

Database interaction: saves new users and accounts, checks duplicate email and phone, loads users during login.

CustomUserDetailsService

Purpose: adapts our database user into Spring Security's expected format.

Why written this way: Spring Security wants a **UserDetails** object, but our app stores users in MySQL. This class bridges that gap.

JwtService

Purpose: generates, parses, and validates JWTs.

Security flow: adds subject, roles, issue time, and expiration, then signs the token using the secret key.

JwtAuthenticationFilter

Purpose: intercepts each request and checks the Authorization header.

Security flow: extracts token, reads username, loads user details, validates token, and sets the authenticated principal in Spring Security's context.

SecurityConfig

Purpose: defines the global rules for API protection.

Frontend-backend communication: also configures CORS so the React app on localhost:5173 can call the backend safely from a different origin.

Mentor note: this module-by-module explanation is exactly the style examiners like, because it proves you understand the role of each class, not just the final outcome.

6. How Login and Protected Requests Work End to End

Login flow example

```
Frontend LoginPage submits email + password
-> Axios POST /api/auth/login
-> AuthController.login()
-> AuthService.login()
-> AuthenticationManager.authenticate()
-> CustomUserDetailsService loads user from MySQL
-> Password check succeeds
-> JwtService.generateToken()
-> token returned to React
-> React stores token and redirects user
```

Protected API example

```
CustomerDashboardPage loads
-> Axios GET /api/customer/account
-> Authorization header contains JWT
-> JwtAuthenticationFilter validates JWT
-> SecurityContext gets authenticated user
-> CustomerController.account()
-> AccountService.getAccountSummary()
-> AccountRepository.findByUser()
-> MySQL returns account
-> DTO returned as JSON
-> React displays account balance
```

Why protected routes matter: without this flow, any user could call admin or customer APIs directly. Security must be enforced on the backend, not just hidden in the UI.

7. Common Viva Questions and Smart Answers

Question	Strong short answer
Why use BCrypt?	Because passwords must never be stored in plain text. BCrypt hashes them.
Why use DTOs in auth responses?	To return only required fields like token, name, email, and roles.
Why use JWT instead of sessions?	JWT supports stateless REST APIs and works cleanly with React clients.
Why reload the user from DB during validation?	Because the server should trust current database roles and status, not only the token.
Why is role-based access important?	Because admin and customer features must be isolated to prevent privilege escalation.

Next, the best continuation is the business module pack: customer module, admin module, transfer logic, transaction internals, and money protection rules.

Use this transition line in a presentation: “Now that I have explained how the system authenticates and authorizes users, I can explain how the banking operations themselves are executed safely.”